

13.2 Pattern-Matching Algorithms

In the classic *pattern-matching* problem, we are given a *text* string of length n and a *pattern* string of length $m \leq n$, and must determine whether the pattern is a substring of the text. If so, we may want to find the lowest index within the text at which the pattern begins, or perhaps *all* indices at which the pattern begins.

The pattern-matching problem is inherent to many behaviors of Java's String class, such as `text.contains(pattern)` and `text.indexOf(pattern)`, and is a subtask of more complex string operations such as `text.replace(pattern, substitute)` and `text.split(pattern)`.

In this section, we present three pattern-matching algorithms, with increasing levels of sophistication. Our implementations report the index that begins the left-most occurrence of the pattern, if found. For a failed search, we adopt the conventions of the `indexOf` method of Java's String class, returning `-1` as a sentinel.

13.2.1 Brute Force

The *brute-force* algorithmic design pattern is a powerful technique for algorithm design when we have something we wish to search for or when we wish to optimize some function. When applying this technique in a general situation, we typically enumerate all possible configurations of the inputs involved and pick the best of all these enumerated configurations.

In applying this technique to design a brute-force pattern-matching algorithm, we derive what is probably the first algorithm that we might think of for solving the problem—we simply test all the possible placements of the pattern relative to the text. An implementation of this algorithm is shown in Code Fragment 13.1.

```

1  /** Returns the lowest index at which substring pattern begins in text (or else -1).*/
2  public static int findBrute(char[] text, char[] pattern) {
3      int n = text.length;
4      int m = pattern.length;
5      for (int i=0; i <= n - m; i++) {                // try every starting index within text
6          int k = 0;                                // k is index into pattern
7          while (k < m && text[i+k] == pattern[k])    // kth character of pattern matches
8              k++;
9          if (k == m)                                // if we reach the end of the pattern,
10             return i;                              // substring text[i..i+m-1] is a match
11     }
12     return -1;                                     // search failed
13 }
```

Code Fragment 13.1: An implementation of the brute-force pattern-matching algorithm. (We use character arrays rather than strings to simplify indexing notation.)

Performance

The analysis of the brute-force pattern-matching algorithm could not be simpler. It consists of two nested loops, with the outer loop indexing through all possible starting indices of the pattern in the text, and the inner loop indexing through each character of the pattern, comparing it to its potentially corresponding character in the text. Thus, the correctness of the brute-force pattern-matching algorithm follows immediately from this exhaustive search approach.

The running time of brute-force pattern matching in the worst case is not good, however, because we can perform up to m character comparisons for each candidate alignment of the pattern within the text. Referring to Code Fragment 13.1, we see that the outer **for** loop is executed at most $n - m + 1$ times, and the inner **while** loop is executed at most m times. Thus, the worst-case running time of the brute-force method is $O(nm)$.

Example 13.1: Suppose we are given the text string

text = "abacaabaccabacabaabb"

and the pattern string

pattern = "abacab"

Figure 13.1 illustrates the execution of the brute-force pattern-matching algorithm on this selection of text and pattern.

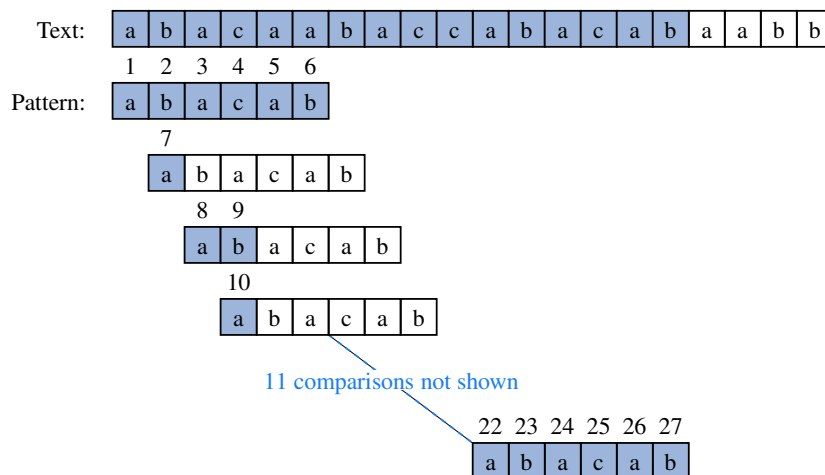


Figure 13.1: Example run of the brute-force pattern-matching algorithm. The algorithm performs 27 character comparisons, indicated above with numerical labels.

The example of Figure 13.2 is rather basic, because it only involves mismatches with the last character of the pattern. More generally, when a match is found for that last character, the algorithm continues by trying to extend the match with the second-to-last character of the pattern in its current alignment. That process continues until either matching the entire pattern, or finding a mismatch at some interior position of the pattern.

If a mismatch is found, and the mismatched character of the text does not occur in the pattern, we shift the entire pattern beyond that location, as originally illustrated in Figure 13.2. If the mismatched character occurs elsewhere in the pattern, we must consider two possible subcases depending on whether its last occurrence is before or after the character of the pattern that was mismatched. Those two cases are illustrated in Figure 13.3.

In the case of Figure 13.3(b), we slide the pattern only one unit. It would be more productive to slide it rightward until finding another occurrence of mismatched character $\text{text}[i]$ in the pattern, but we do not wish to take time to search

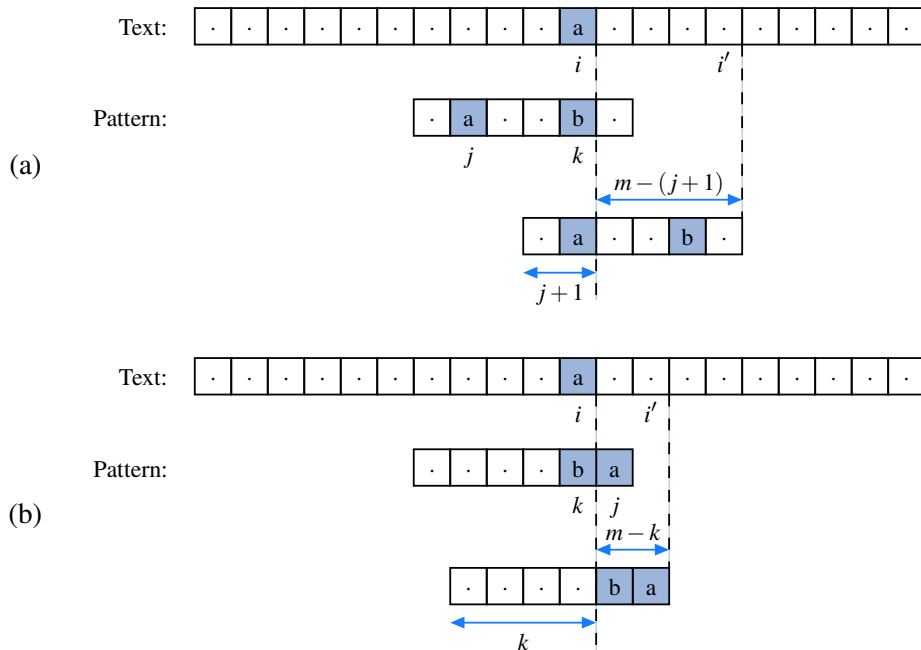


Figure 13.3: Additional rules for the character-jump heuristic of the Boyer-Moore algorithm. We let i represent the index of the mismatched character in the text, k represent the corresponding index in the pattern, and j represent the index of the last occurrence of $\text{text}[i]$ within the pattern. We distinguish two cases: (a) $j < k$, in which case we shift the pattern by $k - j$ units, and thus, index i advances by $m - (j + 1)$ units; (b) $j > k$, in which case we shift the pattern by one unit, and index i advances by $m - k$ units.

for another occurrence. The efficiency of the Boyer-Moore algorithm relies on quickly determining where a mismatched character occurs elsewhere in the pattern. In particular, we define a function $\text{last}(c)$ as

- If c is in the pattern, $\text{last}(c)$ is the index of the last (rightmost) occurrence of c in the pattern. Otherwise, we conventionally define $\text{last}(c) = -1$.

If we assume that the alphabet is of fixed, finite size, and that characters can be converted to indices of an array (for example, by using their character code), the last function can be easily implemented as a lookup table with worst-case $O(1)$ -time access to the value $\text{last}(c)$. However, the table would have length equal to the size of the alphabet (rather than the size of the pattern), and time would be required to initialize the entire table.

We prefer to use a hash table to represent the last function, with only those characters from the pattern occurring in the map. The space usage for this approach is proportional to the number of distinct alphabet symbols that occur in the pattern, and thus $O(\max(m, |\Sigma|))$. The expected lookup time remains $O(1)$ (as does the worst-case, if we consider $|\Sigma|$ a constant). Our complete implementation of the Boyer-Moore pattern-matching algorithm is given in Code Fragment 13.2.

```

1  /** Returns the lowest index at which substring pattern begins in text (or else -1).*/
2  public static int findBoyerMoore(char[] text, char[] pattern) {
3      int n = text.length;
4      int m = pattern.length;
5      if (m == 0) return 0; // trivial search for empty string
6      Map<Character,Integer> last = new HashMap<>(); // the 'last' map
7      for (int i=0; i < n; i++)
8          last.put(text[i], -1); // set -1 as default for all text characters
9      for (int k=0; k < m; k++)
10         last.put(pattern[k], k); // rightmost occurrence in pattern is last
11     // start with the end of the pattern aligned at index m-1 of the text
12     int i = m-1; // an index into the text
13     int k = m-1; // an index into the pattern
14     while (i < n) {
15         if (text[i] == pattern[k]) { // a matching character
16             if (k == 0) return i; // entire pattern has been found
17             i--; // otherwise, examine previous
18             k--; // characters of text/pattern
19         } else {
20             i += m - Math.min(k, 1 + last.get(text[i])); // case analysis for jump step
21             k = m - 1; // restart at end of pattern
22         }
23     }
24     return -1; // pattern was never found
25 }
```

Code Fragment 13.2: An implementation of the Boyer-Moore algorithm.

The correctness of the Boyer-Moore pattern-matching algorithm follows from the fact that each time the method makes a shift, it is guaranteed not to “skip” over any possible matches. For $\text{last}(c)$ is the location of the *last* occurrence of c in the pattern. In Figure 13.4, we illustrate the execution of the Boyer-Moore pattern-matching algorithm on an input string similar to Example 13.1.

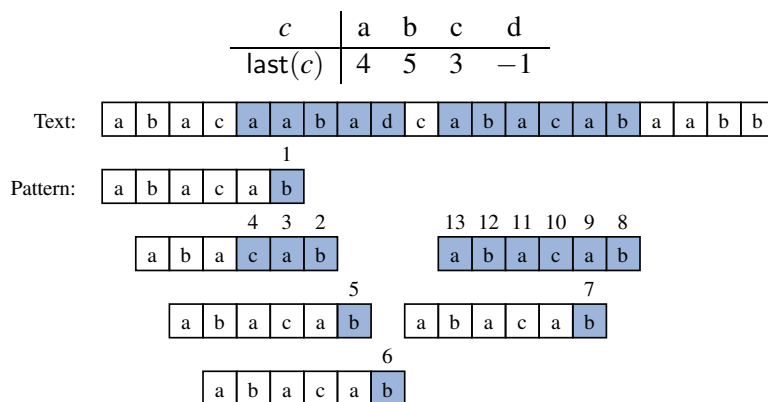


Figure 13.4: An illustration of the Boyer-Moore pattern-matching algorithm, including a summary of the $\text{last}(c)$ function. The algorithm performs 13 character comparisons, which are indicated with numerical labels.

Performance

If using a traditional lookup table, the worst-case running time of the Boyer-Moore algorithm is $O(nm + |\Sigma|)$. The computation of the last function takes $O(m + |\Sigma|)$ time, although the dependence on $|\Sigma|$ is removed if using a hash table. The actual search for the pattern takes $O(nm)$ time in the worst case—the same as the brute-force algorithm. An example that achieves the worst case for Boyer-Moore is

$$\begin{aligned} \text{text} &= \overbrace{aaaaaa \cdots a}^n \\ \text{pattern} &= \overbrace{baa \cdots a}^{m-1} \end{aligned}$$

The worst-case performance, however, is unlikely to be achieved for English text; in that case, the Boyer-Moore algorithm is often able to skip large portions of text. Experimental evidence on English text shows that the average number of comparisons done per character is 0.24 for a five-character pattern string.

We have actually presented a simplified version of the Boyer-Moore algorithm. The original algorithm achieves worst-case running time $O(n + m + |\Sigma|)$ by using an alternative shift heuristic for a partially matched text string, whenever it shifts the pattern more than the character-jump heuristic. This alternative shift heuristic is based on applying the main idea from the Knuth-Morris-Pratt pattern-matching algorithm, which we discuss next.

Example 13.2: Consider the pattern "amalgamation" from Figure 13.5. The Knuth-Morris-Pratt (KMP) failure function, $f(k)$, for the string P is as shown in the following table:

k	0	1	2	3	4	5	6	7	8	9	10	11
$P[k]$	a	m	a	l	g	a	m	a	t	i	o	n
$f(k)$	0	0	1	0	0	1	2	3	0	0	0	0

Implementation

Our implementation of the KMP pattern-matching algorithm is shown in Code Fragment 13.3. It relies on a utility method, `computeFailKMP`, discussed on the next page, to compute the failure function efficiently.

The main part of the KMP algorithm is its **while** loop, each iteration of which performs a comparison between the character at index j in the text and the character at index k in the pattern. If the outcome of this comparison is a match, the algorithm moves on to the next characters in both (or reports a match if reaching the end of the pattern). If the comparison failed, the algorithm consults the failure function for a new candidate character in the pattern, or starts over with the next index in the text if failing on the first character of the pattern (since nothing can be reused).

```

1  /** Returns the lowest index at which substring pattern begins in text (or else -1).*/
2  public static int findKMP(char[] text, char[] pattern) {
3      int n = text.length;
4      int m = pattern.length;
5      if (m == 0) return 0;                                // trivial search for empty string
6      int[] fail = computeFailKMP(pattern);                 // computed by private utility
7      int j = 0;                                           // index into text
8      int k = 0;                                           // index into pattern
9      while (j < n) {
10         if (text[j] == pattern[k]) {                      // pattern[0..k] matched thus far
11             if (k == m - 1) return j - m + 1;             // match is complete
12             j++;
13             k++;
14         } else if (k > 0) {
15             k = fail[k-1];                                // reuse suffix of P[0..k-1]
16         } else {
17             j++;
18         }
19     }
20     return -1;                                           // reached end without match

```

Code Fragment 13.3: An implementation of the KMP pattern-matching algorithm. The `computeFailKMP` utility method is given in Code Fragment 13.4.

Constructing the KMP Failure Function

To construct the failure function, we use the method shown in Code Fragment 13.4, which is a “bootstrapping” process that compares the pattern to itself as in the KMP algorithm. Each time we have two characters that match, we set $f(j) = k + 1$. Note that since we have $j > k$ throughout the execution of the algorithm, $f(k - 1)$ is always well defined when we need to use it.

```

1 private static int[] computeFailKMP(char[] pattern) {
2     int m = pattern.length;
3     int[] fail = new int[m];           // by default, all overlaps are zero
4     int j = 1;
5     int k = 0;
6     while (j < m) {                    // compute fail[j] during this pass, if nonzero
7         if (pattern[j] == pattern[k]) { // k + 1 characters match thus far
8             fail[j] = k + 1;
9             j++;
10            k++;
11        } else if (k > 0)                // k follows a matching prefix
12            k = fail[k - 1];
13        else                             // no match found starting at j
14            j++;
15    }
16    return fail;
17 }
```

Code Fragment 13.4: An implementation of the computeFailKMP utility in support of the KMP pattern-matching algorithm. Note how the algorithm uses the previous values of the failure function to efficiently compute new values.

Performance

Excluding the computation of the failure function, the running time of the KMP algorithm is clearly proportional to the number of iterations of the **while** loop. For the sake of the analysis, let us define $s = j - k$. Intuitively, s is the total amount by which the pattern has been shifted with respect to the text. Note that throughout the execution of the algorithm, we have $s \leq n$. One of the following three cases occurs at each iteration of the loop.

- If $\text{text}[j] = \text{pattern}[k]$, then j and k each increase by 1, thus s is unchanged.
- If $\text{text}[j] \neq \text{pattern}[k]$ and $k > 0$, then j does not change and s increases by at least 1, since in this case s changes from $j - k$ to $j - f(k - 1)$; note that this is an addition of $k - f(k - 1)$, which is positive because $f(k - 1) < k$.
- If $\text{text}[j] \neq \text{pattern}[k]$ and $k = 0$, then j increases by 1 and s increases by 1, since k does not change.

Thus, at each iteration of the loop, either j or s increases by at least 1 (possibly both); hence, the total number of iterations of the **while** loop in the KMP pattern-matching algorithm is at most $2n$. Achieving this bound, of course, assumes that we have already computed the failure function for the pattern.

The algorithm for computing the failure function runs in $O(m)$ time. Its analysis is analogous to that of the main KMP algorithm, yet with a pattern of length m compared to itself. Thus, we have:

Proposition 13.3: *The Knuth-Morris-Pratt algorithm performs pattern matching on a text string of length n and a pattern string of length m in $O(n + m)$ time.*

The correctness of this algorithm follows from the definition of the failure function. Any comparisons that are skipped are actually unnecessary, for the failure function guarantees that all the ignored comparisons are redundant—they would involve comparing the same matching characters over again.

In Figure 13.6, we illustrate the execution of the KMP pattern-matching algorithm on the same input strings as in Example 13.1. Note the use of the failure function to avoid redoing one of the comparisons between a character of the pattern and a character of the text. Also note that the algorithm performs fewer overall comparisons than the brute-force algorithm run on the same strings (Figure 13.1).

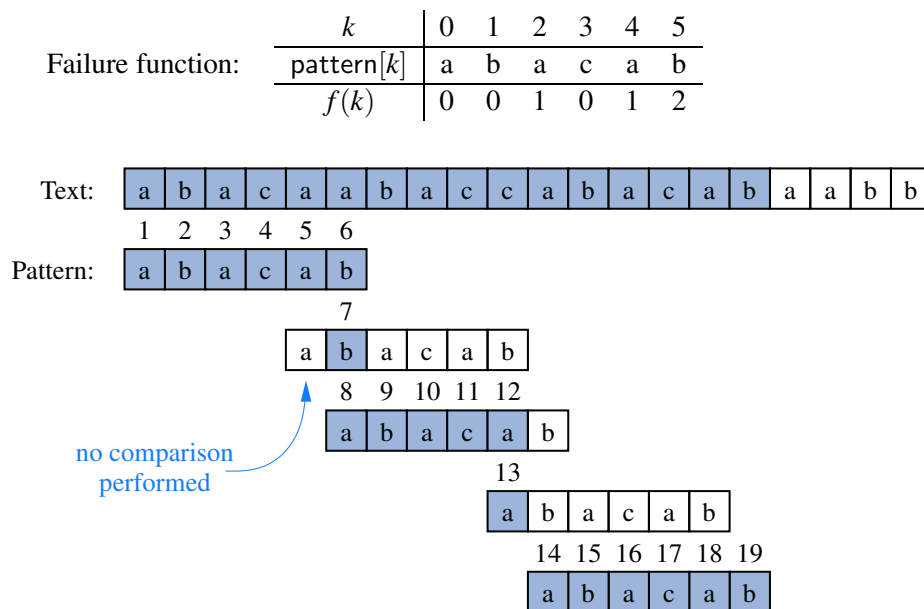


Figure 13.6: An illustration of the KMP pattern-matching algorithm. The primary algorithm performs 19 character comparisons, which are indicated with numerical labels. (Additional comparisons would be performed during the computation of the failure function.)